

AI Advent Challenge #8

Неделя 4. Лекция 4

MCP, инструменты и Tool Orchestration

Отредактированный и дополненный конспект по транскрибации и скринам презентации

Коротко

Неделя посвящена Model Context Protocol: зачем он появился, как через него LLM/агент получает доступ к внешним системам, как устроены MCP host/client/server, tools/resources/prompts, как писать минимальный MCP-сервер и почему для локальной автоматизации иногда выгоднее Skill + CLI, чем MCP.

Что было сделано с исходными материалами

- Сырая транскрибация очищена от повторов, мусора распознавания и разговорных вставок.
- Термин “МСП” приведен к корректному “MCP / Model Context Protocol”.
- Содержимое скринов презентации учтено как визуальный источник: схемы architecture flow, stages, token overhead, comparison MCP vs Skill+CLI и цели недели.
- Отдельно добавлены корректировки: где формулировки лекции слишком грубые, устаревшие или требуют технического уточнения.
- Добавлен практический блок под сдачу недели: MVP MCP playground, структура проекта, сценарий видео и checklist.

1. Контекст недели: зачем нужен MCP

Четвертая неделя курса посвящена инструментарию, а конкретно - Model Context Protocol. На предыдущих неделях уже были разобраны базовые LLM-параметры, управление контекстом, memory, stateful-агенты, state machine и инварианты. Теперь следующий шаг - научить агента безопасно и управляемо работать с внешним миром: календарем, почтой, трекером задач, корпоративной базой знаний, кодом, файловой системой, CRM, внутренними API и другими сервисами.

Базовая проблема проста: LLM обучена на общих данных и сама по себе не знает уникальный контекст конкретной компании, проекта или пользователя. Внутренние знания лежат в Confluence, GitHub/GitLab, Jira/Yandex Tracker, Slack/Telegram-чатах, почте, календарях, базах данных и доменных системах. MCP нужен как стандартный способ подключать эти источники и инструменты к AI-приложениям.

Главный смысл недели

Научиться не просто писать “умные промпты”, а подключать к агенту реальные tools/resources/prompts, оркестрировать их вызовы и понимать цену такого подхода в токенах, безопасности и поддержке.

2. Что такое MCP

MCP расшифровывается как Model Context Protocol. Это открытый протокол для стандартизированной интеграции LLM-приложений с внешними источниками данных и инструментами. Его удобно понимать как “универсальный слой подключения” между AI-приложением и внешними сервисами.

Важная корректировка: MCP - это не конкретный Python/Kotlin/TypeScript-фреймворк. Это протокол и стандарт. У него есть разные SDK и реализации на разных языках, поэтому ощущение “MCP = конкретная библиотека” ошибочно.

Компонент	Роль
MCP Host	AI-приложение, которое управляет одним или несколькими MCP-клиентами. Пример: Claude Desktop, IDE, собственный агентский backend.
MCP Client	Компонент внутри host, который держит соединение с MCP-сервером и получает от него контекст/возможности.
MCP Server	Программа, которая предоставляет tools, resources и prompts. Может быть локальной или удаленной.
LLM	Модель, которая получает контекст, выбирает инструмент, интерпретирует результат и формирует ответ пользователю.

3. Какие задачи решает MCP

- **Доступ к корпоративным знаниям:** Confluence, wiki, внутренние документы, историю чатов, проектные правила, архитектурные решения.
- **Доступ к рабочим системам:** task-трекеры, GitHub/GitLab, CI/CD, CRM, ERP, 1C, базы данных, internal APIs.
- **Доступ к персональным инструментам:** календарь, почта, заметки, файлы, личные TODO, персональные базы знаний.
- **Единый стандарт вместо N интеграций:** не писать для каждой LLM отдельную интеграцию к каждому API, а описать capabilities через MCP.

- **Изоляция возможностей:** можно открыть только безопасные tools, например “прочитать календарь”, но не “создать встречу”.

Практический пример из лекции: руководитель или пользователь может спросить у ассистента, что происходит в бухгалтерии, календаре или трекере задач, не разбираясь в сложном интерфейсе конкретной системы. Агент через MCP получает доступ к нужному сервису, извлекает данные, интерпретирует их и возвращает человеку простой ответ.

4. MCP не заменяет бизнес API

Одна из важных мыслей лекции: до MCP агент мог ходить напрямую в API каждого сервиса. Но API у всех разные: разные форматы ошибок, разные методы, разные HTTP-коды, разные схемы авторизации, разная документация. MCP не отменяет эти API. Обычно MCP-сервер становится “оберткой” над существующим сервисом или API и предоставляет LLM-приложению единый, машинно-читаемый интерфейс.

```
User / UI
-> Host / Agent / Client App
  -> MCP Client
    -> MCP Server (tools / resources / prompts)
      -> External Service / Internal API / DB
    <- Tool result
  -> LLM interprets result
<- Final response
```

5. Что MCP-сервер может отдавать

Primitive	Что это такое	Пример
Tools	Исполняемые функции, которые модель может вызвать через host/client.	get_forecast, read_calendar, create_ticket, query_database
Resources	Данные и контекст, которые можно читать.	файл, запись БД, схема проекта, документ Confluence
Prompts	Переиспользуемые шаблоны взаимодействия.	шаблон для code review, шаблон анализа инцидента, шаблон запроса к CRM
Sampling	Возможность сервера попросить клиент/host выполнить LLM-вызов.	серверу нужен model-independent completion без прямой зависимости от LLM SDK

6. Как выглядит tool schema на примере погоды

На скринах презентации показан пример weather tool: инструмент получает координаты или location, имеет name, title, description, inputSchema и required-поля. Эта схема нужна модели, чтобы понять, какой инструмент существует, когда его выбрать и какие аргументы передать.

```
tool: get_forecast
описание: Get weather forecast for a specific latitude/longitude
inputSchema:
  latitude: number
  longitude: number
required:
  - latitude
  - longitude
```

В Kotlin-примере из презентации MCP-сервер создается через SDK, регистрируется tool, затем внутри обработчика вызывается обычный HTTP-клиент к погодному API. То есть MCP-инструмент сам по себе не магия: это аккуратная обертка над реальным действием или внешним API.

7. Local и remote MCP

Вариант	Как работает	Когда подходит
Local / STDIO	MCP-сервер запускается локально и общается через стандартный ввод/вывод.	Локальные файлы, dev-tools, CLI, IDE, доступ к локальному проекту.
Remote / HTTP	MCP-сервер доступен по сети, обычно через HTTP-транспорт и авторизацию.	Корпоративные сервисы, публичные tools, shared-интеграции, командное использование.

Для публичного MCP-сервера обычно понадобится развернутый backend, домен/URL, HTTPS, авторизация и явно описанные tools/resources/prompts. Для локального демо можно обойтись STDIO-вариантом и локальным SDK.

8. Стадии работы с MCP-инструментом

На слайде со стадиями показан типовой flow взаимодействия агента с MCP. Его важно понимать, потому что именно здесь появляется токеновый overhead.

1. User input: пользователь вводит prompt.
2. Context collection: host собирает system prompt, developer instructions, профиль, state, а также сведения о доступных MCP tools.
3. Sampling call #1: LLM получает контекст и tool schema, выбирает инструмент и аргументы.
4. Tool calling: host/client выполняет JSON-RPC-вызов к MCP-серверу.
5. Sampling call #2: результат инструмента возвращается в контекст модели; LLM интерпретирует результат и формирует ответ.

Важная корректировка

LLM обычно не “сама шлет HTTP-запрос в сервис”. В реальной архитектуре LLM генерирует намерение вызвать tool, а host/client выполняет вызов MCP-сервера и возвращает результат модели. В remote-варианте транспорт может быть HTTP, в local-варианте - STDIO.

9. Плюсы MCP

- **Стандартизация:** один формат интеграций для разных AI-приложений и внешних систем.
- **Переиспользование:** один MCP-сервер можно подключать к разным host/client приложениям.
- **Доступ к реальному миру:** календарь, почта, базы данных, задачи, внутренние API, файлы, Git-репозитории.
- **Изолированные capabilities:** можно дать только конкретные tools, а не весь API целиком.
- **Лучше, чем хаотичный API-wrapper в каждом агенте:** MCP задает общую структуру discovery, schema, call, result.

10. Минусы и риски MCP

В лекции прозвучало, что у MCP “очевидное отсутствие безопасности”. Это полезно как предупреждение, но технически формулировка слишком грубая. Безопасность в MCP - не отсутствует полностью, но MCP действительно открывает опасные поверхности атаки: tool poisoning, prompt injection, over-permission, небезопасные CLI-команды, утечки данных и нежелательные действия.

Риск	Что может случиться	Как снижать риск
Over-permission	Tool может делать больше, чем нужно: читать лишние данные, создавать/удалять сущности.	Least privilege: давать только нужные tools, разделять read/write, требовать confirmation.
Prompt injection	Внешний контент может заставить модель вызвать опасный tool.	Фильтрация, sandbox, explicit allowlist, validation, human confirmation.
Command injection	Tool вызывает shell/CLI с неочищенными аргументами.	Не передавать raw input в shell, использовать allowlist команд, escaping, sandbox.
Data leakage	Модель получает слишком много приватного контекста.	Scope, access control, masking, audit logs.
Tool result confusion	Tool вернул неожиданный результат, а LLM интерпретировала его неверно.	Строгая схема результата, typed responses, post-validation.

11. Token overhead: почему MCP может быть дорогим

Вторая крупная тема лекции - сравнение MCP с подходом Skill + CLI. Главный тезис: MCP удобен как стандарт, но может быть дорогим по токенам, особенно когда у сервера много tools, tool schema большая, а агент делает много последовательных действий.

На слайдах приведен пример: MCP Server с 46 tools имеет tool schemas примерно на 5 787 токенов и загружается always при sampling call; Skill + CLI имеет SKILL.md, references и bash schema суммарно около 4 723 токенов, но может грузиться on demand, а idle cost может быть 0 токенов. Эти числа не универсальны: они относятся к конкретной реализации и конкретному набору инструментов.

Подход	Сильная сторона	Слабая сторона
MCP	Единый стандарт, удобно для remote/shared/public tools, discovery, interoperability.	Tool schemas/results могут раздувать контекст и стоимость при частых вызовах.
Skill + CLI	Хорошо для локальной автоматизации, dev-workflow и повторяющихся действий; можно грузить on demand.	Нужен локальный CLI/tool, сложнее стандартизировать для внешних пользователей и remote-сервисов.

12. Важная корректировка: MCP не “deprecated” в общем смысле

В транскрибации звучит сильная формулировка, что MCP “становится deprecated”. В учебном контексте это лучше понимать не буквально, а как тезис о неэффективности MCP для некоторых локальных и повторяющихся сценариев, где Skill + CLI может быть дешевле и проще.

Корректная формулировка

MCP не стоит считать устаревшим стандартом. Он по-прежнему полезен для стандартизированных remote/shared integrations. Но для локальной автоматизации, тестирования, CLI-команд и тяжелых

итеративных workflow Skill + CLI может быть заметно экономнее по токенам.

13. Когда выбирать MCP, а когда Skill + CLI

Ситуация	Лучший выбор	Почему
Нужно подключить корпоративный сервис к разным AI-клиентам	MCP	Стандартный протокол, переиспользование, remote access, discovery.
Нужно дать Claude/Cursor/агенту локально управлять проектом или тестами	Skill + CLI	Меньше token overhead, можно вызывать локальные команды on demand.
Нужно работать с календарем, почтой, CRM, трекером задач	MCP	Нужна стандартизированная интеграция и контроль доступов.
Нужно 20-50 раз подряд дергать инструмент в рамках одного dev-flow	Skill + CLI или hybrid	MCP-схемы и история вызовов могут дорого раздувать контекст.
Нужно сделать публичный инструмент для других пользователей	MCP	Пользователь не обязан иметь локальный CLI; подключение идет через стандартный server interface.
Нужно сравнить стоимость и точность	Оба	Именно это полезно как эксперимент недели: замерить токены и качество на одинаковой задаче.

14. Оркестрация MCP

В рамках своего агента важно не просто “подключить один tool”, а уметь оркестрировать набор tools. Запрос пользователя может потребовать несколько действий: прочитать календарь, проверить задачи, найти свободные окна, создать черновик встречи, запросить подтверждение и только потом записать событие.

```
User query
-> intent detection
-> select relevant MCP tools
-> call tool #1: read calendar
-> call tool #2: read task tracker
-> LLM reasoning over results
-> if destructive action: ask confirmation
-> call tool #3: create event / update task
-> validate result
-> final response
```

Здесь снова возвращаются темы предыдущих недель: state machine, invariants, validation, allowed transitions и memory. MCP без stateful-архитектуры быстро превращается в хаотичный набор вызовов.

15. Практический MVP для недели

Для сдачи недели можно сделать небольшой MCP playground. Не нужно сразу строить сложный enterprise-server. Важно показать, что ты понял протокол, можешь поднять server, описать tool schema, подключить client/host, вызвать tool, получить результат и сравнить это с альтернативой Skill + CLI.

Компонент	Что сделать
MCP server	Минимальный сервер с 1-2 tools: get_weather, get_time, read_notes, search_tasks или аналог.
Tool schema	У каждого tool должны быть name, description, input schema, required params.
Tool call	Показать реальный вызов инструмента и получение результата.
Agent/host layer	Собрать prompt, передать tool schema, выбрать tool, интерпретировать результат.
Logging	Логировать user query, selected tool, arguments, result, final answer.
Comparison	Сделать аналогичный сценарий через Skill + CLI или хотя бы расчет token overhead.
README	Описать запуск, архитектуру, ограничения и выводы.

16. Рекомендуемая структура проекта

```

ai-advent-week4-mcp-playground/
├── README.md
├── pyproject.toml
├── .env.example
├── src/
│   ├── mcp_server.py
│   ├── mcp_client.py
│   ├── agent.py
│   ├── prompt_builder.py
│   ├── token_meter.py
│   └── tools/
│       ├── weather.py
│       └── notes.py
├── skills/
│   └── weather_skill.md
├── results/
│   ├── mcp_run.md
│   ├── skill_cli_run.md
│   └── comparison.md
└── demo/
    └── script.md

```

17. Сценарий видео для сдачи

- Показать README и коротко объяснить: это MCP playground для недели 4.
- Показать MCP server: какие tools доступны и какая input schema у каждого tool.
- Запустить сервер и client/agent.
- Сделать запрос пользователя, например: “дай прогноз по координатам” или “найди заметку по теме”.
- Показать, что агент выбрал tool, передал аргументы, получил result и сформировал нормальный ответ.
- Показать логи: tool name, arguments, result, final answer.
- Показать сравнение с Skill + CLI или хотя бы объяснить вывод по token overhead.
- В конце проговорить: где MCP полезен, где Skill+CLI дешевле, какие риски безопасности учтены.

18. Что было на слайдах презентации

- Страница 1:** пример JSON tool schema для weather_current и Kotlin-фрагмент создания MCP Server со STDIO transport.

- **Страница 2:** Kotlin data models/client для погоды и tool get_forecast с latitude/longitude, inputSchema и проверкой required-полей.
- **Страница 3:** архитектурная схема User/UI -> Host/Agent/Client App -> MCP Server -> LLM и отдельная пометка, где токены не тратятся, а где начинается token spending.
- **Страница 4:** stages MCP-flow: user input, context collection, sampling call #1, tool calling JSON-RPC, sampling call #2; также сравнение token overhead MCP Server vs Skill+CLI.
- **Страница 5:** таблица масштабирования токенов для MCP vs Skill+CLI и цели недели: познакомиться с MCP, реализовывать для публичных tools, оркестрировать, использовать в реальной работе, сравнить Skill+CLI.

19. Корректировки и замечания к лекции

Фрагмент/идея из транскрибации	Корректировка
“МСП”	Это ошибка распознавания. Правильно: MCP / Model Context Protocol.
“MCP встраивается в нейронку”	Точнее: MCP работает между host/client и server. Модель не обязательно напрямую общается с MCP-сервером; обычно вызовы исполняет host/client.
“Любая LLM умеет общаться с MCP”	Точнее: MCP должен поддерживать host/client layer. Самой LLM нужен инструментальный слой, который преобразует tool calls в MCP-вызовы.
“LLM просто шлет HTTP-request”	Не всегда. MCP поддерживает разные транспорты, включая local STDIO и remote HTTP. Вызов обычно делает приложение-хост, а не сама модель.
“MCP без безопасности”	Слишком грубо. У протокола есть authorization/security considerations, но tool-доступ действительно требует least privilege, sandbox, confirmation, validation и audit.
“MCP deprecated”	Слишком сильное утверждение. Корректнее: для локальных повторяющихся workflow Skill+CLI может быть дешевле по токенам; MCP остается полезным стандартом для shared/remote integrations.
Token таблицы из слайдов	Полезны как демонстрация, но числа зависят от конкретной схемы tools, количества вызовов, клиента, модели и реализации. Не стоит считать их универсальным законом.

20. Цели недели

- Познакомиться с Model Context Protocol и понять его место в архитектуре AI-агентов.
- Научиться реализовывать MCP для публичных или внешних tools.
- Научиться оркестрировать несколько MCP-вызовов в собственном агенте.
- Попробовать применить MCP в реальной рабочей задаче или максимально близком демо.
- Потестировать сравнение MCP и Skill + CLI: по токенам, удобству, скорости, надежности и безопасности.

21. Итоговая формула лекции

Итог

MCP - это стандартный способ дать агенту инструменты и контекст из внешнего мира. Но подключение tools само по себе не делает агента хорошим. Нужны state machine, ограничения, безопасность, оркестрация, validation и понимание token economics. MCP полезен для shared/remote integrations; Skill+CLI часто выгоднее для локальной агентной автоматизации.

22. Источники и что именно использовано

- Пользовательская транскрибация: Nueva grabacion 2m4a (2).txt - основа лекции и логика повествования.
- Пользовательский PDF со скринами презентации: 1.pdf - схемы architecture, stages, token overhead, MCP vs Skill+CLI и цели недели.
- Официальная документация Model Context Protocol: architecture overview и specification - использованы для корректировки терминов host/client/server, JSON-RPC, tools/resources/prompts, transports и security considerations.
- Официальная публикация Anthropic "Introducing the Model Context Protocol" - использована для уточнения происхождения и назначения MCP.